How a Proof Becomes a Program

A visual tour of a small formal system in which theorems, once proved, hand you a working program
— and of what that turns out to cost.

Ara Aslyan

Prologue

In school we learn that a proof is something that convinces us a statement is true. A proof of "every even number greater than two is a sum of two primes" would settle the matter; it would not, on the face of it, *do* anything.

Computer scientists discovered something stronger. Sometimes a proof does not merely establish that an object exists — it contains, inside itself, a recipe for *building* that object. A proof that "every list can be sorted" can hide an actual sorting algorithm in its steps. When that happens, the proof is not just an argument; it is a program in disguise, and with the right lens you can read the program straight off the proof. Proof assistants — software that checks proofs down to the last symbol — let us apply that lens *mechanically*, turning a verified proof into runnable code with no human cleverness in between.[1]

For simple statements this is unsurprising. The interesting question is what happens at the other extreme. Some theorems need reasoning far stronger than ordinary arithmetic — reasoning that climbs past the natural numbers into the infinite, using principles a first course in induction never mentions. When a proof reaches that high, does the program still fall out the bottom? Or does the extra strength leave a residue — some step that asserts an object exists without ever building it — that quietly hollows the extracted program into something that type-checks but cannot actually run?

This document answers that question by opening one such extraction pipeline from end to end. We take a fixed, small formal system; we prove inside it a theorem that genuinely needs more than ordinary induction; we watch, in full detail, the proof become a program; and we then ask the proof assistant, which cannot lie, exactly which assumptions that program depends on. The tour is meant to be read start to finish by someone who has never seen any of this before. Everything is built up on the page, and each idea arrives at the moment an earlier paragraph has made you ask for it.

[1] The development this tour describes is carried out in Lean 4 [1] with its mathematical library Mathlib [11]; the extensional-collapse layer it builds on is a companion Kleene–Kreisel continuous-functionals development. Throughout, "the proof assistant" and "the machine" mean this system.

Act I — The hook

Pick a natural number — say 3. Write it down. Now play the following game with it.

First, rewrite the number using only powers of 2, and rewrite the exponents that way too, and their exponents, all the way down, until nothing but 2s and small constants remain. This is called *hereditary base* 2. Now change every 2 you wrote into a 3 — bump the base — and subtract 1 from the result. Then rewrite *that* number in hereditary base 3, change every 3 into a 4, subtract 1 again. Then base 4 into base 5, subtract 1. And so on, bumping the base by one each step and subtracting a single unit each time.

The bases are marching upward without bound. Each step replaces a base by a larger one everywhere it appears — and near the start, that base change inflates the number far more violently than the lonely "subtract 1" ever pulls it back down. The sequence you are generating does not drift gently; for many starting values it erupts. Start at 4 and the first terms are those in Figure 1: the number climbs into values with no short decimal name — and yet, eventually, it comes back down to 0.

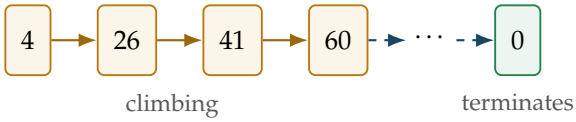

Figure 1: The Goodstein sequence starting at 4: it erupts upward, driven by the base change, then — after astronomically many steps — reaches exactly 0. (The sequence starting at 3 is tame by comparison: $3, 3, 3, 2, 1, 0$.)

Goodstein's theorem [3] says that this sequence — no matter which number you started from — always, eventually, hits exactly 0.

That is already strange. A process whose every early move makes the number enormously bigger, driven downward only by subtracting one at a time, nonetheless terminates at 0 for *every* starting value. But the strange part, for our purposes, is not the theorem itself. It is what it takes to prove it, and what you can do with the proof once you have it.

Here is the first half of the surprise. You cannot prove Goodstein's theorem with ordinary mathematical induction over the natural numbers. The usual "check 0, then check that each number's truth follows from the previous one" is genuinely not enough; the theorem needs a stronger principle, one that reaches past where step-by-step induction on $\mathbb{N}$ can go. (Exactly what that stronger principle is, and why it is stronger, is Act IV's business. For now, hold onto the fact that ordinary induction falls short.)

Here is the second half. Despite needing that extra strength, the proof is *constructive* — it does not merely assert that the sequence reaches 0, it contains, buried inside it, a recipe for finding the exact step at which that happens. And in the formal system this tour is about, that recipe can be pulled out of the proof mechanically and run as an actual program. Feed it a starting number; it returns the stopping step.

Most expositions of Goodstein's theorem stop at the statement, or gesture at the proof and leave the "constructive content" as a phrase. This one opens the box all the way. We are going to watch, in complete detail and on a real example, a proof turn into a program — first on something tiny enough to hold in your hand, then on Goodstein itself, and then on a handful of other theorems that each ask the machine to do one genuinely new thing. By the end you will have seen the whole mechanism, and one question will remain: when a program falls

out of a proof this powerful, does some sliver of non-constructive reasoning sneak into the program along the way? The last act is devoted to answering it.

Act II — One tiny, real example, walked through completely

Before any general theory, one concrete thing, done slowly and in full. We will not introduce a single piece of machinery in this act beyond what this one example forces us to name. No hierarchies, no levels, no collapses — those are Act III, and they will make far more sense once you have a real realizer in hand to point at.

Everything in this tour happens inside a fixed, small formal system — we will call it *the fragment*. Think of the fragment as a self-contained little world of mathematics with a rigid rulebook: a fixed vocabulary of things it can talk about (zero, successor, addition, multiplication, and a modest list of further operations we meet as we need them), a fixed grammar of statements it can form, and a fixed list of inference rules for building proofs. Nothing enters the fragment except through that rulebook. This rigidity is deliberate: because the rules are finite and mechanical, a proof in the fragment is itself a finite, mechanical object — a tree of rule-applications — and a machine can take that tree apart. Hold that thought; it is what makes everything later possible.

Our example is the fragment's very first existential theorem — the first time it proved that *something exists* rather than that two things are equal. It is deliberately the warm-up the development itself used before tackling the general Goodstein theorem, and we use it for exactly that reason: in Act IV, Goodstein will be the grown-up version of this same statement, and you will already have watched the miniature happen by hand. The statement is: *there is a step s at which the Goodstein sequence starting from 3 has reached 0*, written in the fragment's grammar as

$$\exists s.\ \mathrm{good}(3, s) = 0,$$

where $\mathrm{good}(3, s)$ is the fragment's own name for "the value of the Goodstein sequence started at 3, after s steps."

We happen to know the answer. The Goodstein sequence starting from 3 runs $3, 3, 3, 2, 1, 0$, so it first reaches 0 at step 5. The number 5 is the *witness*: the particular s that makes $\exists s.\ \mathrm{good}(3, s) = 0$ true. Keep 5 in mind; the whole act is about watching it travel from the proof into a running program and back out.

What a realizer for this has to be

Here is the first idea we genuinely need, and it arrives because the statement forces it. The statement claims something *exists*. A proof that merely convinces us the claim is true — without ever saying *which* s works — would be useless for computing anything. So we ask: what would a proof have to carry, concretely, for us to read the answer off it?

At a minimum, it has to carry the witness — the number 5 — because without it there is nothing to return. And it has to carry, as well, some evidence that this witness really does the job: that $\mathrm{good}(3, 5)$ really is 0. This "what a proof carries" is exactly the notion of a **realizer**: the computational content of a proof, the data a machine can actually use. The shape of that data is dictated by the statement's outermost connective, and for an existential it is precisely the pair we just argued for:

> *A realizer of $\exists y.\ \varphi(y)$ is a pair x: a **witness** $\pi_1 x$, and a **certificate** $\pi_2 x$ that realizes φ at that witness. Compactly,*
>
> $$x \Vdash \exists y.\ \varphi(y) \quad \Longleftrightarrow \quad \pi_2 x \Vdash \varphi(\pi_1 x).$$

Read $\pi_1 x$ as "read the witness off x" and $\pi_2 x$ as "the rest of x." This is not a metaphor chosen for this paper; it is the actual clause the fragment uses to define what "realizes" means at an existential.

Now specialize it to our formula. Instantiating the clause at $\exists s.\ \mathrm{good}(3, s) = 0$ gives

$$x \Vdash \exists s.\ \mathrm{good}(3, s) = 0 \quad \Longleftrightarrow \quad \pi_2 x \Vdash \big(\mathrm{good}(3, \pi_1 x) = 0\big).$$

The body $\mathrm{good}(3, s) = 0$ is an *equation*, and here a second small idea appears, again forced by the example. What does it take to realize an equation? An equation is either true or not; if it is true, there is nothing further to *compute* about it — no witness to choose, no case to decide. So the realizer of a true equation carries no information at all: any object realizes it, provided the equation holds. We call such a realizer **contentless**. So the certificate $\pi_2 x$ above is contentless, and the realizer of $\exists s.\ \mathrm{good}(3, s) = 0$ is, in effect, *just the witness* 5 wearing a trivial second component. All of the content is the witness. That is as simple as a realizer ever gets, which is why we started here.

The proof, and what extract does to it

Now we build an actual proof of $\exists s.\ \mathrm{good}(3, s) = 0$ in the fragment, in the shape the realizer analysis predicts. To prove an existential the fragment has a rule — existential introduction — that says: to conclude $\exists y.\ \varphi$, supply a specific term for y and a proof that φ holds for that term. We supply the term 5. That leaves us needing a fragment-internal proof that $\mathrm{good}(3, 5) = 0$ — and this the fragment can simply *compute*, grinding its defining equations for good on the concrete numbers 3 and 5 until it reaches 0. So the whole proof is: existential-introduction, at 5, over the computed equation, a small tree with the witness 5 at the branch point.

The fragment comes with a function — extract — that takes a finished proof and returns its realizer, by walking the proof tree and doing, at each rule, the small operation that rule's realizer calls for. On our proof it does exactly what the shape predicts (Figure 2): the witness 5 the rule supplied becomes the pair's first component; the realizer of the computed equation — contentless — becomes the second. And to run the result, we read its first component at the canonical point, recovering 5.

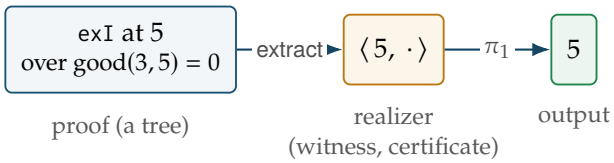

Figure 2: The whole round trip on the tiny example. The proof already contains the witness 5; extract only repackages it into the standard existential shape ⟨witness, certificate⟩, the certificate contentless because the body is an equation. Reading the first component returns 5.

That is the full round trip on the smallest possible real example: a statement was formed; we reasoned out what a proof of it must carry; we built such a proof by hand; extract repackaged its witness; evaluating the result returned 5. Nothing here required knowing what a "level"

is, where realizers "live," or how any of this scales. Those questions push themselves on us the moment we ask "does this work for *every* proof?" — and Act III introduces exactly the machinery they demand.

Act III — Opening the box

We have watched one proof become one program. Two questions are now unavoidable. First: our example's body was an equation, whose realizer was contentless — so *this* realizer was essentially just a number. Real theorems have bodies with "and," "or," implications, "for all." What are realizers of *those*? Second: we ran extract once and got the right answer. Is that luck, or does extract produce a correct realizer for *every* proof? We take the questions in that order, because the second needs the first.

Realizers, for every shape of statement

In Act II we met the realizer clause for $\exists$ by asking what a proof of an existential must carry. Every connective has such a clause, arrived at the same way, and once you have seen one the rest are recognizable rather than surprising.[2] Table 1 collects them — the guiding question each time being *what would a machine need to use a proof of this?*

Statement	Realizer	to use a proof of it, you. . .
$s = t$	(nothing)	. . . need nothing — it just has to hold
$A \wedge B$	a pair (r_A, r_B)	. . . get at each side
$A \vee B$	a tag + a payload	. . . first learn which side
$A \to B$	a function $r_A \mapsto r_B$	. . . feed it evidence of A
$\forall x.\, A(x)$	a function $k \mapsto r_{A(k)}$	. . . pick an x
$\exists x.\, A(x)$	a pair (witness, certificate)	. . . read the witness

Table 1: What realizes each shape of statement. An equation is *contentless*. A conjunction holds both realizers at once; a disjunction tags which side holds (0 left, non-zero right) and carries that side's realizer — structurally the same "a number, then a payload it selects" as the existential, which is why the fragment builds $\exists$ out of the disjunction's pairing machinery. The two that *consume* an input — the functions of $\to$ and $\forall$ — are the ones that do work at runtime.

You have now seen the shape of every realizer the fragment will ever produce. Two of them — implication and universal — take something in and give something back; they are the procedures, the parts that *do work* at runtime. To use a proof of "if A then B" you feed it evidence of A and it returns evidence of B; to use "for all x" you pick an x and it returns the instance. The other three — pair, tag, witness — just *hold* data. Keep that distinction in view; the next section — levels — is built entirely on it.

[2] The interpretation is Kreisel's *modified* realizability [7]; the standard reference, whose induction-axiom realizer this development follows, is Troelstra [12].

Levels: the cost of consuming

Here is the question the previous paragraph forced. Implications and universals are procedures — they take an input and return an output. But that input is *itself* a realizer, which might itself be a procedure taking an input, and so on. A realizer can be a procedure whose inputs are procedures whose inputs are procedures. How deep does the nesting go, and how do we track it?

The fragment tracks it with a single number on each formula, its **level** ℓ, counting how many layers of "takes an input, returns an output" a realizer can involve. The counting rule is short, and it is exactly the consume-versus-hold distinction made arithmetic:

$$\begin{aligned} \ell(s = t) &= 0, & \ell(A \to B) &= \max\big(\ell A + 1,\ \ell B\big), \\ \ell(A \wedge B) &= \max\big(\ell A, \ell B\big), & \ell(\forall x.\, A) &= \ell A + 1, \\ \ell(A \vee B) &= \max\big(\ell A, \ell B\big), & \ell(\exists x.\, A) &= \ell A. \end{aligned}$$

Only the two consuming connectives add a level. Why does $\forall$ cost a level but $\exists$ does not, when both mention a bound variable? Because $\forall y.\, \varphi$ is a *procedure* — you hand it a y and it works — while $\exists y.\, \varphi$ merely *holds* a y, the witness, already chosen. A universal consumes; an existential packages. Consuming costs a level; packaging is free. Look back at Act II: the witness 5 was already sitting in the realizer, waiting to be read; nothing had to be *run*. That is exactly why our existential example was so simple: $\exists$ adds no level. The rule is not imposed from outside — it falls straight out of the difference between a realizer that runs and one that merely stores.

Where a realizer lives: PureType

We keep saying a realizer "is a number," or "is a procedure taking a realizer and returning a realizer." Once realizers can be procedures whose inputs are procedures, we must say precisely what mathematical objects these are. Now that Act II has given us a concrete realizer, and levels have told us how deep the nesting goes, we can place them. Realizers live in a tower of types, built by that same level count:

$$\mathsf{PureType}_0 = \mathbb{N}, \qquad \mathsf{PureType}_{n+1} = \big(\mathsf{PureType}_n \to \mathbb{N}\big).$$

The bottom floor is the natural numbers — where a bare witness like our 5 lives. Each floor up is the functions from the floor below to the numbers (Figure 3). Each time a formula's level rises by one — each time you add an implication or universal that *consumes* a realizer — the realizer climbs one storey: a realizer of a level-n formula lives n storeys up. This is why we spent so long on levels first: the level is the floor number.

The program a closed statement denotes: CtQ

Two *different* proofs of the same statement can produce two different realizers — different objects in the tower — that nonetheless compute the same answers everywhere. To speak of "*the* program" a theorem denotes, we collapse realizers that agree as functions into single objects. For a closed statement, its realizer lands, after this collapse, in a space the development calls CtQ, and its image there is the one *proof-independent* program the theorem denotes:

$$\boxed{\text{Realizer}} \xrightarrow{\text{collapse}} \boxed{\text{CtQ}}.$$

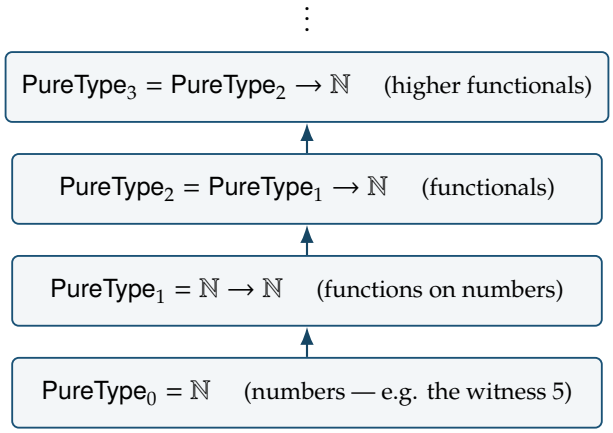

Figure 3: The PureType tower. A realizer of a level-n formula lives on floor n. Our Act II realizer was a level-0 object — a number — because its formula, an existential over an equation, adds no level. This tower is built by completely elementary means, with nothing classical or non-constructive anywhere in it — a fact Act V will turn on.

We name CtQ now but do not yet lean on it: for the collapse to be the honest space of functionals rather than an arbitrary quotient, the realizers landing in it must be *continuous*, a property we state below. Read CtQ for now as a promissory note — *the program of a closed theorem is a single proof-independent object, once we know its realizer is continuous* — redeemed two definitions on.[3]

Soundness: it works for every proof

Now the second opening question. We ran extract once and got a correct realizer. Is that always so? It is — and it is the central theorem of the whole construction:

> ***Soundness.*** *For every proof the fragment can build, of every statement,* extract *returns a genuine realizer of that statement.*

What we watched happen once, by hand, for $\exists s.\ \mathrm{good}(3, s) = 0$, soundness guarantees for *every* derivation the fragment admits. Its proof is itself an induction — not over numbers, but over the shape of proofs. There are finitely many inference rules; soundness checks, once, that each rule's piece of extract does the right thing *assuming the pieces for its sub-proofs did*. Because every proof is a finite tree of those rules, checking each rule once covers every proof there will ever be. From here on we may speak of "the program a theorem extracts to" and know, without further checking, that it is correct at every input.

Generic continuity: every program is continuous

One more property, stated now and explained more deeply once Act IV gives us programs worth explaining it for. A realizer high in the tower is a functional — it takes functions as inputs. The pathological ones read *infinitely much* of their input before deciding an output, and those are exactly the objects that do not collapse honestly into CtQ. The good functionals are

[3] CtQ is the extensional collapse of the Kleene–Kreisel continuous functionals [6, 7]; for a modern account see Longley and Normann [8].

the **continuous** ones: to determine any single output they consult only *finitely much* of their input. The development proves, once and generically:

> ***Generic continuity.*** *Every realizer* extract *produces, from every proof, is continuous.*

This redeems the promissory note on CtQ: because every extract is continuous, every closed theorem's realizer collapses honestly into a single CtQ element — one honest program. We will see *why* generic continuity holds in Act VI; the harder examples of Act IV should exist first to make that "why" worth the space. The box is open: we have realizers for every shape, the level accounting, the tower they live in, the collapse into programs, and the two guarantees — correctness and continuity. Everything from here runs on exactly this machinery, unchanged. What varies is only the proofs we feed it.

Act IV — Running the same machine on harder proofs

Nothing new gets added to the extractor in this act. The pipeline — prove a theorem, run extract, get a certified program — is fixed. What changes is the difficulty of the proofs, and each example is chosen because it forces exactly one genuinely new capability out of the same fixed machine. We name that capability in a heading, so the staircase is visible (Figure 4). Before the first step, one rule the first five examples all lean on.

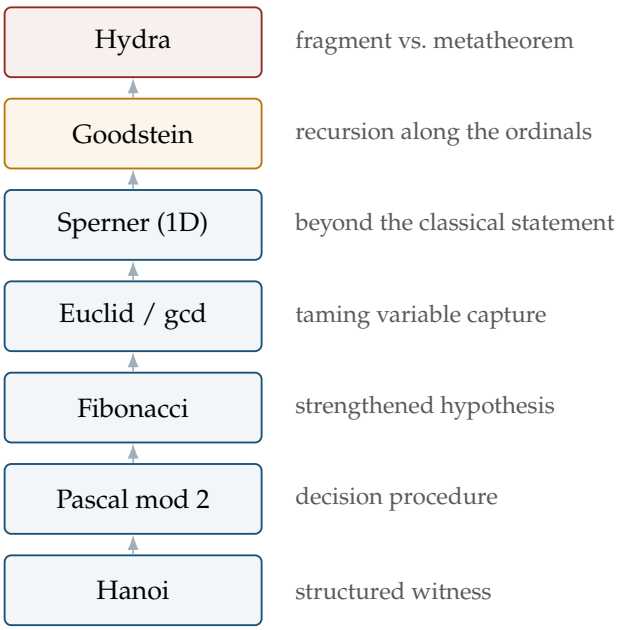

Figure 4: The staircase of Act IV: each example is the one below plus a single new demand on the same fixed extractor. Amber marks the step up to transfinite recursion (Goodstein); red marks the boundary of what the fragment can even express (Hydra).

The rule that becomes recursion: ordinary induction

The fragment has ordinary mathematical induction as an inference rule — call it `ind`. It says the familiar thing: to prove $\forall x.\, \varphi(x)$, prove $\varphi(0)$ and prove that $\varphi(x)$ implies $\varphi(x+1)$. Watch what extract does to a proof that uses it. The base $\varphi(0)$ extracts to a realizer — the answer at 0. The step extracts to a *transformation* — a procedure turning a realizer of $\varphi(x)$ into one of $\varphi(x+1)$. To get the realizer at a number k, extract starts from the base and applies the step k times:

$$R(0) = r_0, \qquad R(n+1) = f\big(R(n)\big).$$

That is precisely a recursive program: a base value and a function iterated as many times as the input says. A proof by induction, run through extract, *is* a program by recursion (Figure 5) — the shape underneath all five of the next examples.

Figure 5: Extraction turns the base and step of an induction into the base and step of a recursion. The stronger induction of Goodstein (below) turns, by the same move, into a stronger recursion.

New capability: a structured witness (Towers of Hanoi)

The Towers of Hanoi puzzle asks for a sequence of moves transferring n disks between pegs. The fragment proves such a sequence always exists — and, in the same breath, that it has exactly $2^n - 1$ moves. What is new is the *shape* of the witness. In Act II the witness was a bare number, 5. Here the thing whose existence we prove is a whole *sequence of moves* — structured data, encoded as a number but meaning a list. When extract runs, the realizer's witness is that move sequence, and it decodes into a readable list of "move a disk from this peg to that peg" instructions — the classical optimal solution, produced by the extracted program and checked to be exactly the moves a human would write. The existential machine of Act II, unchanged, now returns structured data. (The proof uses `ind` on the disk count, so it extracts to a recursion; the recursion branches — two smaller sub-towers joined by one move — which is why the count comes out $2^n - 1$.)

New capability: a decision procedure (Pascal's triangle, mod 2)

Replace every entry of Pascal's triangle by its remainder mod 2: every number becomes a 0 or a 1. The fragment proves, for each position, that its parity is 0 or 1 — and, more to the point, *which*. The new capability is that the extracted program is a **classifier**. The statement is a disjunction, "this entry is 0, or it is 1," so (Table 1) its realizer carries a tag naming the answer. Run extract and that tag becomes a runnable function from a position to its parity — a decision procedure, produced not by us writing one but by extracting it from a proof that merely said "the answer is one or the other." Tabulate that classifier's outputs — a mark for 1, a blank for 0, row by row — and the picture that appears is the Sierpiński triangle. There is no gap between the picture and the theorem: every cell of the drawing is one instance of the proved disjunction, its value filled in by the extracted realizer's own tag.

New capability: a strengthened induction hypothesis (Fibonacci)

The Fibonacci numbers $0, 1, 1, 2, 3, 5, 8, \ldots$ are each the sum of the previous two. Suppose we want to prove, in a form from which we can extract a program, that fib is total. Try it by ordinary induction and you hit a wall worth feeling: induction hands you the statement *at* n, but computing $\mathrm{fib}(n+2)$ needs *two* earlier values, $\mathrm{fib}(n+1)$ and $\mathrm{fib}(n)$, and the hypothesis at n alone does not give both. The new capability is the standard cure. Instead of proving "$\mathrm{fib}(n)$ exists," prove the stronger pair statement: the *pair* $\big(\mathrm{fib}(n), \mathrm{fib}(n+1)\big)$ exists. Now the hypothesis hands you both consecutive values, and the step is a single shift,

$$\big(\mathrm{fib}(n),\ \mathrm{fib}(n+1)\big) \longmapsto \big(\mathrm{fib}(n+1),\ \mathrm{fib}(n) + \mathrm{fib}(n+1)\big).$$

A stronger hypothesis is a stronger thing to be handed at the step, so the induction goes through. Extract that proof and — exactly as Figure 5 predicts — you get the fast two-variable iterative Fibonacci every programmer knows: carry a pair, shift it n times, read off a component. The proof's strengthened hypothesis *is* the program's pair of accumulator variables; you can watch the extracted pair advance through $(0,1), (1,1), (1,2), (2,3)$ — the loop's running state, read out of a proof that never mentions a loop.

New capability: taming variable capture (Euclid's gcd)

The fragment proves that any two numbers have a greatest common divisor — and the extracted witness *is* the gcd, computed by subtractive Euclid (repeatedly replace the larger number by the difference), with no gcd operation built into the fragment. Two supporting economies make this possible. First, the fragment never added an "order" operation: $s < t$ is simply $\exists d.\, (s+1) + d = t$ — order is an existential over addition, not a primitive. Second, it never added "strong induction" (where the step may use *all* smaller cases); that is *derived* from ordinary `ind`.

But the new capability the gcd proof forces is neither of these — it is a problem about *substitution*. When you instantiate an induction hypothesis or a "for all," you substitute a term for a bound variable; if that term mentions a variable that is *also* bound in the formula, the substitution silently confuses two different variables sharing a name — the notorious *variable capture*. The fragment's substitution is deliberately simple-minded: rather than quietly renaming to avoid capture, it *forbids* the capturing substitution outright, so the proof author must route around it. Euclid's recursion is the first place two capture problems bite at once, and the first to use the fragment's two general-purpose dodges *together*: give the offending term a fresh name with an extra "for all" and instantiate at the harmless name; and, when a recursion permutes its arguments, rename those variables to fresh ones first. Neither changes what is proved or what the program computes; both merely coax a naive substitution system into accepting a substitution that is morally fine. Out the far end comes a certified greatest-common-divisor calculator — which, pleasingly, needs no special case for zero: the specification is met by $\gcd(0,0) = 0$, since everything divides 0.

New capability: more general than the classical statement (Sperner, 1D)

Colour the points $0, 1, \ldots, n$ along a line, each some natural-number colour, with the two ends coloured differently. Then somewhere two *adjacent* points must differ — you cannot get from one end-colour to the other without a change. This is the one-dimensional Sperner lemma [9], the discrete ancestor of the intermediate value theorem. The fragment proves it by ordinary

induction, scanning the line and carrying "either we are still at the start colour, or we have already found a crossing"; the extracted program is a left-to-right search returning the *first* crossing. But the new thing is not the extraction — it is the theorem. The classical statement is usually given for *two* colours; the fragment's proof never uses that restriction. It works for *arbitrary* natural-number colourings, so the extracted search is more general than the classical picture. The proof turned out to prove *more* than the theorem it was written for.

New capability: recursion along the ordinals (Goodstein)

Now Goodstein itself, the general theorem behind Act II's single hand-worked case. Act II proved $\exists s.\ \mathrm{good}(3, s) = 0$ with the witness supplied by us as 5. Goodstein's *theorem* is the universal version,

$$\forall m.\ \exists s.\ \mathrm{good}(m, s) = 0,$$

and we want from it the same thing Act II gave, but uniform in m: a program that, given m, returns the stopping step. The proof cannot know m's answer in advance, so the witness must be *produced* by the proof.

This is where Act I's promissory note — that ordinary induction is not enough — comes due. Induct on m, or s, or the value $\mathrm{good}(m, s)$, and you fail: the value goes *up* at almost every step, so no natural-number quantity decreases as the sequence runs. What decreases is not a natural number at all. To see why, write a number in hereditary base and watch a step act on it:

$$13 \;=\; 2^{2+1} + 2^2 + 1 \;\xrightarrow{2 \rightsquigarrow 3}\; 3^{3+1} + 3^3 + 1 \;=\; 109 \;\xrightarrow{-1}\; 108.$$

Replacing every base 2 by a 3 — *including the ones inside the exponents* — inflates 13 to 108 in a single step. Yet if you read that same expression not as a number but as an *ordinal* — interpret each base as ω, the first infinity — the base change does nothing (every base is already ω), while the "-1" strictly *lowers* it. Every Goodstein step, however it inflates the natural-number value, strictly descends an ordinal below ε_0 (epsilon-nought); and the ordinals are *well-ordered* — no infinite descending chains (Figure 6) — so the sequence cannot descend forever and must reach 0.

$$0 < 1 < 2 < \cdots < \omega < \omega + 1 < \cdots < \omega^2 < \cdots < \omega^{\omega} < \cdots < \varepsilon_0.$$

This is the extra strength Act I warned of: **transfinite induction up to** ε_0, the single rule in the fragment's whole repertoire that is not a consequence of ordinary logic and ordinary induction — it is exactly the principle Gentzen showed lies beyond ordinary arithmetic [2], here realized as recursion along a primitive-recursive ordering in the manner of Tait [10]. To make it mechanical the fragment builds the ε_0 ordinals *as ordinary numbers* under a hand-crafted encoding, defines the "smaller ordinal" relation on those codes, and proves — by elementary means, no infinite reasoning — that it admits no infinite descending chain. (Why the encoding was built by hand rather than borrowed is an Act VI discovery.)

Now the two acts line up. In Act II, `extract` turned an existential proof into a witness by reading a number the proof supplied. Here it turns Goodstein's proof into a witness by *recursion along the ordinals*: the transfinite-induction rule extracts to a program that descends the ordinal of each state until it reaches the bottom, and the number of descents is the stopping step. Where `ind` extracted to "iterate a step k times" (Figure 5), the transfinite rule extracts to "recur along the ordinal ordering" — a stronger recursion for a stronger induction. Same

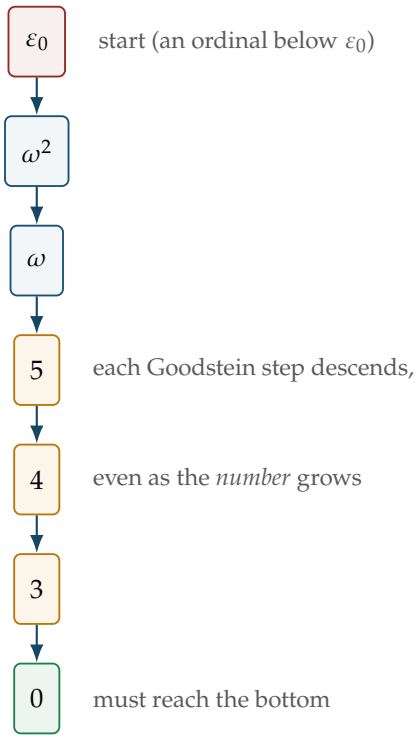

Figure 6: Well-foundedness in one picture: the ordinal assigned to each state strictly descends at every step, and a descending chain of ordinals cannot go on forever, so the process terminates — however large the underlying numbers become.

machine, one gear higher:

$$\forall m.\ \exists s.\ \mathrm{good}(m, s) = 0 \qquad \xrightarrow{\text{extract}} \qquad m \longmapsto s.$$

One theorem; one extracted program. And it runs (Table 2). The row for $m = 4$ is where Act I's "numbers explode" stops being an assertion: the program declines to finish because the terminating sequence really is that long.

start m	0	1	2	3	$\cdots$	4
stopping step s	0	1	3	5	$\cdots$	(not attempted)

Table 2: The extracted stopping-step program $m \mapsto s$. It returns 0 and 1 directly when run; soundness certifies 3 and 5 for $m = 2, 3$ (the sequences $2, 2, 1, 0$ and $3, 3, 3, 2, 1, 0$) even where running the program that far is slow; $m = 4$ is not attempted — its sequence begins $4, 26, 41, 60, \ldots$ and its stopping step is astronomically large.

New capability: a fragment theorem versus a metatheorem (the Hydra)

The last and hardest example marks a boundary — the place where what the fragment can *prove* and what it can even *state* come apart.

The Hydra is a many-headed tree; Hercules chops a head and it regrows, often growing *more* heads than it lost. The Kirby–Paris theorem [5] says Hercules wins anyway: every battle

terminates. The reason is Goodstein's in disguise — each chop, though it grows the tree, strictly lowers an ordinal below ε_0 assigned to it. The fragment can prove termination for *one fixed, encoded battle*: fix a starting Hydra and a fixed way of playing, code it all as numbers, and prove $\forall h.\ \exists t.\ \mathrm{hydra}(h, t) = 0$ — the exact shape of Goodstein's theorem, proved the exact same way, extracting to a battle-length program. Business as usual.

But the *real* Kirby–Paris theorem says Hercules wins *no matter what strategy* he uses — and this the fragment cannot even express (Figure 7). To say "for every strategy" you must quantify over strategies, and a strategy is a *function* (it looks at the current Hydra and decides the next chop). The fragment's "for all" ranges only over *numbers*, never functions. The limitation is not that the fragment fails to *prove* the general theorem; the sentence expressing it is not in the fragment's language at all. So the general theorem is proved *outside* the fragment, in the ordinary mathematics of the proof assistant, which quantifies over functions freely; and the fragment's single battle is checked to be one of those general plays, a genuine special case. This split — a *fragment theorem* for one battle, a *metatheorem* for all strategies — is forced, cleanly and provably, by the one fact that the fragment has no function variables. It is the right place to stop feeding the machine harder proofs.

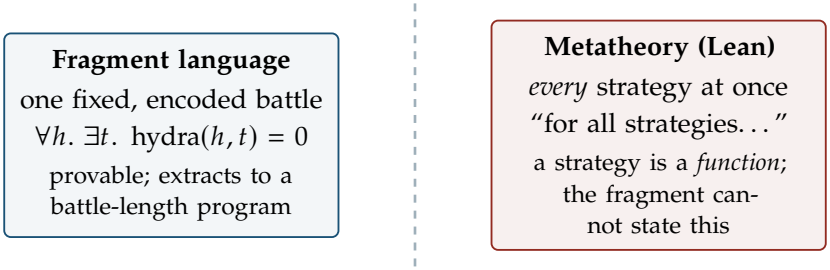

Figure 7: The boundary the Hydra draws. Left of the dashed line, the fragment proves — and extracts a program for — termination of one encoded battle. Right of it, the fully general claim, which the fragment cannot even write down, so it is proved directly in the metatheory. The line is forced by the absence of function variables, not chosen.

Act V — What it costs: the constructivity audit

We have now watched proofs of real strength become running programs. Goodstein and the Hydra needed transfinite induction up to ε_0 — genuinely more than ordinary arithmetic. And the programs are not toys: an ordinal-descending recursion, a first-crossing search, a greatest-common-divisor calculator, a parity classifier.

So here is the suspicion any careful reader should now be holding, and it is the right one. Proofs of this strength, in ordinary practice, routinely use *non-constructive* reasoning — the law of excluded middle, the axiom of choice, arguments that assert an object exists without building it. Surely, somewhere in a construction reaching all the way to ε_0, some non-constructive step sneaks in. And if it does, the extracted "program" might be a fiction — an object that type-checks but cannot run, its content secretly resting on an axiom that computes nothing. The whole promise of the tour would collapse at its most impressive point.

This act resolves the suspicion, precisely, in the program's favour — and the resolution is possible only because the proof assistant tracks, mechanically and unforgeably, exactly

which axioms any object depends on. The suspicion has a specific name. The genuinely non-constructive axiom — the one that asserts choices can be made with no rule for making them, and from which the full law of excluded middle follows — is called `Classical.choice`. It is the axiom whose presence in a program would hollow it out, turning a thing that computes into a thing that merely type-checks. So the question has one precise form: does `Classical.choice` appear in the list?

Now the finding, stated exactly as the machine reports it. Every extracted program in the whole development — the Goodstein stopping-step function, the Hydra battle-length function, the gcd calculator, the Sperner search, the Pascal classifier, the Fibonacci iterator, and the extraction machinery itself — reports its axiom dependency as exactly

```
[propext, Quot.sound]
```

and `Classical.choice` is *not* on the list. Not "we believe"; not "morally." The kernel, asked directly, lists those two names and not the third, for every object that actually runs.

So what *are* the two names that are there? Neither is the one the suspicion feared. `propext` and `Quot.sound` are the proof assistant's own two foundational axioms, and both are constructively harmless: neither lets you conjure an object you cannot build, neither implies excluded middle. They are bookkeeping about when two propositions, or two quotient elements, count as equal — part of the machine's floor, computationally inert. The trusted base of every extracted computation is the kernel together with these two, and nothing classical.

Does `Classical.choice` appear *anywhere*? Yes — and where it does is a precise boundary (Table 3, Figure 8). It enters in exactly two places, and neither runs. It appears in some *correctness proofs* — the proofs that the programs meet their specifications — and even there only through two humble library lemmas about updating a function at a point; these live entirely among propositions and are used to *prove a program correct*, never inside it. And it appears in the final *packaging* step — the collapse of a realizer into its single CtQ element — performed once to say "here is the one program this theorem denotes," inheriting choice from the general theory of extensional functionals. But again: the packaging is *about* the program; the program itself is the choice-free realizer that got packaged.

Component	**Uses `Classical.choice`?**
Extracted programs (Goodstein, gcd, . . .)	**no**
The extraction pipeline itself	**no**
Some correctness proofs	limited — two library lemmas
The CtQ packaging step	yes

Table 3: Where the one non-constructive axiom does and does not appear. Everything that runs is choice-free; `Classical.choice` is confined to things that only reason *about* the programs.

A proof needing strictly more than Peano arithmetic — transfinite induction to ε_0 — became a program, and the program is *constructive to the last axiom the machine can name*. The strength went into proving the theorem; none of it leaked into the thing that runs.

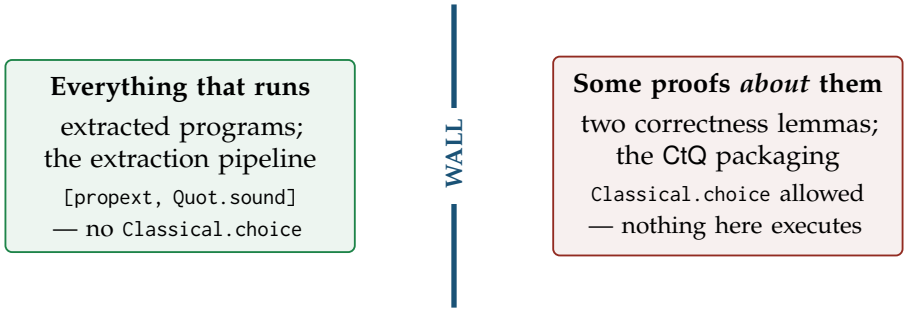

Figure 8: The constructivity wall. On one side, everything that executes — choice-free, resting only on the inert kernel axioms. On the other, some proofs *about* those programs and one packaging step, which may use choice freely because nothing there ever runs. The one non-constructive axiom in the whole development is quarantined on the far side from anything that computes.

Act VI — What the machine forced us to discover

A proof assistant is usually cast as a checker: you have the argument, it confirms you made no mistake. That is the smaller half of what happened here. The larger half is that the machine, by refusing to accept certain things, *forced* decisions that would never have surfaced on paper — where a wave of the hand carries you past exactly the spots where it dug in. Four episodes; each is a place where the development is shaped the way it is because the machine would not let it be otherwise.

The level bookkeeping was not designed — it was extorted

Act III's realizers live in a tower of types, and each formula carries a level saying how high in the tower its realizers sit. The rule turned out to be sharp:

> *Connectives that* consume *a realizer — implication and "for all" — cost a level. Connectives that merely* package *realizers — "and," "or," and "there exists" — cost nothing.*

On paper one might guess a level discipline and tune it until the proofs go through. Here there was no room to guess: assign "there exists" a level it should not have, or deny "for all" the level it needs, and the realizer types fail to line up and the type-checker rejects the extractor outright — not with a warning, with a hard error at the exact malformed clause. The rule above was read off the pattern of what the machine accepted and what it refused. The distinction it encodes — consuming versus packaging — is a real conceptual fault line in realizability, and it was the type-checker that drew it.

The Hydra boundary was provable, not merely felt

Act IV ended at the wall where the general Kirby–Paris theorem cannot be stated in the fragment. On paper that is easy to slide past — one writes "for every strategy. . . " and reads it as innocent. Inside the machine it is not innocent: to write it you must produce a term quantifying over functions, and the fragment's grammar has no such term former. The attempt does not produce a weak or unprovable statement; it produces *no statement at all*, because it does not parse. The split between the fragment theorem (one battle) and the metatheorem (all strategies) is therefore not a matter of taste about where to stop — it is forced by a checkable

property of the fragment's syntax. The machine turned "this feels out of reach" into "this is provably inexpressible."

The pairing had to be hand-built, or the audit would have failed

Act V's clean verdict — nothing that runs uses `Classical.choice` — was nearly lost to a detail three layers down. The ε_0 ordinals are encoded as ordinary numbers, and that encoding needs a way to pack two numbers into one: a pairing function. The proof-assistant library ships one, ready to use. But its supporting lemmas are, throughout, proved with `Classical.choice` — and because the transfinite recursor's very *definition* calls the pairing, that choice would have flowed downstream into extract and tainted *every* extracted program (Figure 9). The paper argument "we encode ordinals as numbers" hides this completely; `#print axioms`, run on an early prototype, made it glaring — `Classical.choice` appearing in a program that should have had none, traced back to the borrowed pairing. The fix was to hand-roll a triangular pairing that the kernel can compute and whose lemmas are choice-free. A foundational decision, invisible on paper, forced into the open by the audit the machine made possible.

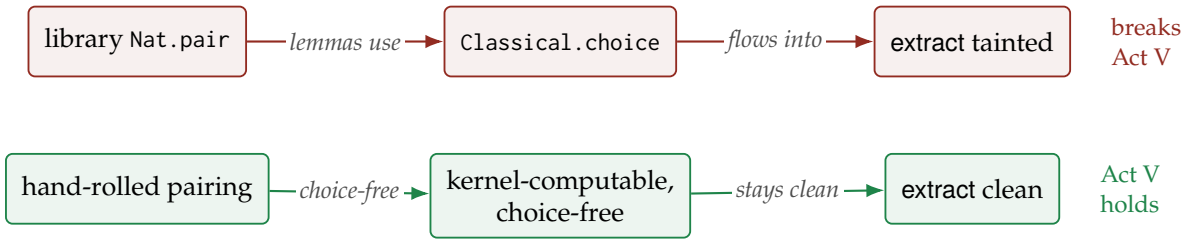

Figure 9: The discovery the audit forced. Reusing the library's pairing (top) would have threaded `Classical.choice` through the ordinal recursor into every extracted program. Hand-rolling a choice-free pairing (bottom) keeps the one non-constructive axiom out of everything that runs. The leak was invisible on paper and glaring under `#print axioms`.

A verified optimization that changed nothing — and we kept it

The transfinite recursor is the development's most expensive machine, so an obvious improvement suggests itself: *memoize* it — cache each ordinal's result so it is not recomputed. This was implemented, proved correct, and wired in as a drop-in replacement for the plain recursor. Then it was measured, and it changed *nothing*. The reason is precise and only visible once you look: producing the recursor's value at an ordinal code is already a constant-time operation — it hands back a closure — so the real cost lives in the *applications* of that closure, which a table keyed by the ordinal code can never reach. The verified-but-useless memo path is kept in the development, with its correctness proof, as the record of a negative result that was *measured* rather than assumed. On paper an optimization that looks plausible tends to get written up as a win; here the profiler said otherwise, and honesty about the null result was cheaper than the illusion of a speedup.

The descent was demoted from assumption to theorem

The engine of Goodstein and the Hydra is one fact: each step strictly lowers the ordinal. In an early version that fact was simply *assumed* — imported as an axiom schema and used. That

is a real dependency, and the machine records it: an assumed descent appears on the `#print axioms` list, in plain sight, for anyone to see how much is being taken on faith. The development later *earned* it back, deriving the descent from three small schemas, each about a single symbol in relation to a single ordinal operation, none mentioning Goodstein or the Hydra at all. The gain is not that the proof got shorter — it is that the ledger of assumptions got smaller, and the machine is what keeps that ledger honest. What you assume is visible; reducing it is progress you can point to, not a matter of narrative.

Act VII — The thesis

Set the examples back-to-back — Hanoi, Pascal, Fibonacci, Euclid, Sperner, Goodstein, the Hydra — and the natural summary is "look, one pipeline turns proofs into programs." But that pipeline is old news: that a constructive proof carries computational content is the content of realizability and of the Curry–Howard correspondence [4], both decades established. The reader was told as much in Act I, and being shown a known correspondence at work is not, by itself, a surprise.

The surprise is the *range reached with so little*. One signature of twenty-one symbols. One extractor, with one small combinator per inference rule. One soundness proof, one case per rule. One continuity invariant, closed under the same handful of combinators. That fixed, modest apparatus — never enlarged across the whole tour — was enough to reach a structured puzzle solution, a decision procedure that draws the Sierpiński triangle, a strengthened-invariant iterator, a greatest-common-divisor calculator, a discrete intermediate-value search, and two theorems — Goodstein and Kirby–Paris — that genuinely need transfinite induction to ε_0, more than ordinary arithmetic can prove. Wildly different mathematics; the same few moving parts.

And the whole way down, the machine kept the accounting honest — not as a formality but as the thing that made the strongest claim sayable at all. The constructivity verdict of Act V is not an aspiration one hopes holds up under scrutiny; it is a fact the kernel will reprint on demand, for every program in the development: everything that runs rests on two inert axioms and nothing classical. A proof that needs more than Peano arithmetic became a program that needs less than a whiff of choice — and we know it needs less because the machine, asked directly, said so.

That is how a proof becomes a program: not by a grand mechanism, but by a small one applied with discipline, and checked by something that cannot be talked past.

References

[1] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37.

[2] Gerhard Gentzen. Beweisbarkeit und unbeweisbarkeit von anfangsfällen der transfiniten induktion in der reinen zahlentheorie. *Mathematische Annalen*, 119:140–161, 1943. doi: 10.1007/BF01564760.

[3] R. L. Goodstein. On the restricted ordinal theorem. *The Journal of Symbolic Logic*, 9(2): 33–41, 1944. doi: 10.2307/2268019.

[4] William A. Howard. The formulae-as-types notion of construction. In J. P. Seldin and J. R. Hindley, editors, *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, pages 479–490. Academic Press, 1980. Written 1969.

[5] Laurie Kirby and Jeff Paris. Accessible independence results for Peano arithmetic. *Bulletin of the London Mathematical Society*, 14(4):285–293, 1982. doi: 10.1112/blms/14.4.285.

[6] Stephen C. Kleene. Countable functionals. In A. Heyting, editor, *Constructivity in Mathematics*, pages 81–100. North-Holland, 1959.

[7] Georg Kreisel. Interpretation of analysis by means of constructive functionals of finite types. In A. Heyting, editor, *Constructivity in Mathematics*, pages 101–128. North-Holland, 1959.

[8] John Longley and Dag Normann. *Higher-Order Computability*. Theory and Applications of Computability. Springer, 2015. doi: 10.1007/978-3-662-47992-6.

[9] Emanuel Sperner. Neuer beweis für die invarianz der dimensionszahl und des gebietes. *Abhandlungen aus dem Mathematischen Seminar der Universität Hamburg*, 6:265–272, 1928. doi: 10.1007/BF02940617.

[10] William W. Tait. Functionals defined by transfinite recursion. *The Journal of Symbolic Logic*, 30(2):155–174, 1965. doi: 10.2307/2270132.

[11] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020. doi: 10.1145/3372885.3373824.

[12] A. S. Troelstra, editor. *Metamathematical Investigation of Intuitionistic Arithmetic and Analysis*, volume 344 of *Lecture Notes in Mathematics*. Springer, 1973. Modified realizability and its soundness: Ch. III, §3.4.